

**UNIVERSITY OF MINDANAO TAGUM BRANCH BACHELOR OF SCIENCE IN
INFORMATION TECHNOLOGY**

DEPARTMENT OF COMPUTER EDUCATION

Visayan Village, Tagum City, Philippines



WashWise: A Mobile Laundry Pickup, Scheduling, and Order Tracking System

A TITLE PROPOSAL FOR CCE 106/L

SUBMITTED TO:

PRINCESS ANNE DADUL

SUBMITTED BY:

CAGAS, ROEWIN LUKE R.

ILAHAS, HANNCE KRISTAN V.

GADIOSO, CARL PATRICK

SEPTEMBER 2026

TABLE OF CONTENTS

CHAPTER 1	4
1.1 Introduction	4
1.2 Project Overview	6
1.3 Requirements	9
CHAPTER 2	11
2.1 Architecture	11
2.2 Design	12
2.3 Development Process	16
CHAPTER 3	18
3.1 Testing.....	18
3.2 Deployment	19
3.3 Maintenance	23
CONCLUSION	24
REFERENCES	25

LIST OF TABLES

Table 1	Project information.....	6
Table 2	Functional requirements	9
Table 3	Nonfunctional requirements	10
Table 4	Technology stack	11
Table 5	Firestore collection schema.....	13
Table 6	Proposed API operations.....	15
Table 7	Key test cases and expected outcomes.....	18
Table 8	Deployment environments.....	20

LIST OF FIGURES

Figure 1	Proposed full development Gantt chart	16
----------	---	----

CHAPTER 1

INTRODUCTION PROJECT OVERVIEW AND REQUIREMENTS

1.1 Introduction

Purpose and Scope of the Document

This document presents the requirements, architecture, design, development process, testing, deployment, and maintenance plan for WashWise. It guides the development and evaluation of a mobile application for laundry ordering, branch selection, processing visibility, ready notifications, and in-person settlement. Customers, staff, and administrators have separate responsibilities under the updated project workflow.

Intended Audience

The intended audience consists of the project adviser and evaluators, the student development team, laundry shop administrators and staff, and customer representatives. Technical sections guide implementation, while the requirements and workflow explain the expected behavior to users.

Background of the Study

Laundry services are commonly used by students, employees, families, and other individuals who may not have enough time to wash and manage their clothes themselves. However, customers often need to visit or contact a laundry shop to inquire about services, submit their laundry, and check the progress of their orders. This may cause inconvenience, waiting time, and communication problems, especially when customers need to repeatedly ask about the status of their laundry.

The proposed WashWise mobile application will provide customers with a convenient way to create laundry orders, select between Regular and Rush/Express services, view an estimated price, and monitor the progress of their laundry. After submitting an order, customers will personally bring their dirty laundry to the laundry shop, where staff will verify the actual laundry and determine the final price. When the laundry is ready and the customer returns to collect it, the customer will pay the laundry shop in person. The system will not process online payments or provide delivery services. Instead, laundry staff will manage the processing schedule based on the selected service and current workload, while customers can monitor the status of their laundry through the application.

Related studies support the use of mobile applications to improve laundry services. Leong et al. (2020) presented a laundry management and on-demand mobile application with laundry ordering, tracking, and notification functions. Nurmayang Sari and Widiono (2025) examined mobile tracking for laundry services to improve transparency and operational efficiency. Rosli et al. (2026) further explored an on-demand laundry logistics platform with real-time tracking and customer support. These studies support the relevance of the proposed WashWise application and its focus on laundry ordering, service management, status tracking, and customer notifications.

The updated workflow distinguishes customer self-registration from staff provisioning. Only the administrator creates staff accounts, using an email address and temporary password through a protected backend. Ready notification precedes payment and collection. An order becomes completed only after staff record the payment and confirm pickup.

Statement of the Problem

General Problem

How can a mobile application make laundry ordering and status tracking more convenient for customers while helping staff manage orders and processing schedules?

Specific Problems

1. How can customers create orders and select Regular or Rush/Express services?
2. How can the system provide estimated prices and completion times?
3. How can staff verify laundry details, final prices, and in-person payments?
4. How can customers monitor their laundry status?
5. How can the system notify customers when laundry is ready?
6. How can staff manage orders, priorities, and processing statuses?
7. How can customers browse and select their preferred laundry shop branch and view its location through GPS and Google Maps?

Objectives

General Objective

To develop a mobile application that allows customers to order laundry services, view estimates, track progress, and receive notifications while helping staff manage orders and processing.

Specific Objectives

1. To allow customers to register, log in, and place orders.
2. To provide Regular and Rush/Express services.
3. To calculate estimated prices and completion times.
4. To allow staff to verify laundry details, final prices, and payments.
5. To allow customers to track laundry status.
6. To allow staff to manage processing priorities and statuses.
7. To notify customers when laundry is ready for pickup.
8. To maintain order and transaction history.
9. To allow customers to browse and select from multiple laundry shop branches, and to view branch locations and directions through GPS and Google Maps integration.

1.2 Project Overview

Table 1 Project information

Item	Description
Application name	WashWise: A Mobile Laundry Pickup, Scheduling, and Order Tracking System
Current version	1.0.0 in the supplied project configuration. This label does not establish a production release or client acceptance.
Expected release	End of Week 8 of the proposed development schedule, subject to successful testing and client acceptance. Calendar dates are to be agreed with the adviser and client.
Development team	CAGAS, ROEWYN LUKE R.; ILAHAS, HANNCE KRISTAN V.; GADIOSO, CARL PATRICK
Academic stakeholder	PRINCESS ANNE DADUL; Department of Computer Education, University of Mindanao Tagum Branch
Operational stakeholders	Laundry customers, branch staff, the laundry shop administrator, and the client representative.

In the project title, pickup means the customer collects finished laundry at the selected shop. Scheduling refers to staff-managed processing order and Regular or Rush/Express priority. It does not mean laundry delivery or customer-controlled collection reservations.

Target Users

The proposed application will be used by:

Primary Users: Laundry customers

Secondary Users: Laundry shop staff

Administrator: Laundry shop administrator authorized to manage staff access, branches, and pricing.

Main Features

The mobile application will include:

User Registration/Login - Allows customers to register and securely access their accounts. Staff and administrators log in using authorized accounts; staff cannot register themselves.

User Profile – Allows users to view and manage their personal information.

Customer Dashboard – Displays laundry shop branches for the customer to choose from, along with current orders, laundry status, and important updates.

Branch Selection – Allows customers to browse laundry shop branches, each displayed with a photo, name, description, and address, and select a branch before placing an order.

GPS and Google Maps Integration – Allows customers to view a selected branch's exact location and get directions through Google Maps.

Laundry Order Placement – Allows customers to submit their laundry details and create an order.

Regular and Rush/Express Service Selection – Allows customers to choose their preferred laundry service based on their needs.

Estimated Price Calculation – Calculates the estimated laundry cost based on the provided laundry details.

Estimated Completion Time – Provides an estimated time when the laundry may be completed.

Laundry Status Tracking – Allows customers to monitor the progress of their laundry order.

In-Person Payment Recording – Allows staff to record the customer's payment after paying at the laundry shop.

Notifications – Alerts customers about important order updates, especially when their laundry is ready for pickup.

Order History/Records – Allows customers and staff to view previous laundry orders and transaction records.

Staff/Admin Management - Allows staff to manage assigned branch orders, processing priorities, payments, and laundry statuses. The administrator manages staff accounts, branches, and prices and monitors operational records.

Scope and Limitations

Scope:

The system will focus on laundry order management and status tracking across multiple laundry shop branches. After logging in, customers are directed to the Home/Dashboard, where they can browse available branches, each shown with a photo, name, description, and address, and select their preferred branch to start an order. Customers can view a branch's exact location and get directions through GPS and Google Maps integration. Once a branch is selected, customers can create orders, select services, view estimated prices and completion times, monitor laundry progress, and receive notifications. Staff can verify laundry details, confirm final prices, record in-person payments, manage processing priorities, and update order statuses.

Limitations:

The system will not include online payment, laundry delivery, customer-controlled pickup scheduling, or automated control of laundry machines. Customers must personally bring their dirty laundry to the shop, pay in person, and return to the shop to pick up their completed laundry. Estimated prices and completion times may change after staff verifies the actual laundry and current workload.

Authentication and live data updates require an internet connection. Push notifications depend on device permission and delivery services, so the application also retains an in-app notification and the current order status. Estimates are advisory and may change with verified laundry details and branch workload.

Expected Output

The expected output is a functional mobile application that will help laundry customers conveniently place orders, select services, view estimates, track laundry progress, and receive ready-for-pickup notifications while helping laundry staff manage orders, payments, processing schedules, and laundry statuses.

1.3 Requirements

Functional Requirements

Table 2 Functional requirements

ID	Required behavior
FR01	Customers register, log in, recover access, and maintain their own profiles. Registration cannot grant staff or administrator privileges.
FR02	Customers browse branch details, prices, and locations, select a branch, and open map directions.
FR03	Customers choose service and Regular or Rush/Express priority, enter laundry details, and confirm an estimate. Submission creates an owned order with pending_dropoff status.
FR04	Authorized branch staff inspect the laundry, verify actual weight or items, confirm the final price, and change the order to received.
FR05	Staff update received to washing to drying to ready; customers can view only their own order progress.
FR06	The ready transition creates a customer notification and initiates a push alert before payment and collection.
FR07	Staff verify the in-person payment against the final amount and confirm collection before setting completed.
FR08	Customers view their own completed history; staff view authorized branch records; administrators monitor operational records.
FR09	Only an administrator creates staff using email and temporary password. The backend sets role=staff, createdBy, createdAt, and isActive=true.
FR10	Administrators deactivate/reactivate staff and manage branches and prices. Deactivation preserves the account and audit records.
FR11	Authorized cancellation records cancelled status and an optional reason under the shop-approved policy without erasing payment or history.

ID	Required behavior
FR12	Orders, payments, notifications, and account administration retain ownership, branch, actor, and timestamp information for traceability.

Nonfunctional Requirements

The following are proposed acceptance targets to be measured during testing, rather than results already achieved.

Table 3 Nonfunctional requirements

Area	Acceptance target
Performance	Common reads complete within 3 seconds in at least 95% of 30 trials on the agreed device and stable network; a confirmed status change appears in a second connected session within 5 seconds. Record test conditions.
Security	Deny cross-customer access, unauthorized branch updates, role escalation, and customer payment edits. Require active staff/admin authorization for privileged operations. Passwords remain only in Firebase Authentication.
Usability	Use clear labels, consistent navigation, readable forms, and visible validation. Representative users complete their assigned core scenario without a critical error; record assistance needed.
Reliability	Retries do not create duplicate orders, payments, or ready notifications. Failed requests never display false success, and pending saves do not overwrite the last confirmed state.
Data integrity	Validate quantities, branch ownership, monetary values, and state transitions. Use server timestamps and preserve the pricing basis used by an order.
Compatibility	Evaluate customer and operational screens on selected Android devices. Additional platform releases require separate configuration and verification.
Maintainability	Separate screens, service logic, and data access. Use readable TypeScript types, reviewed changes, versioned rules, and repeatable tests.
Recovery	Back up operational data before risky changes and verify restoration in staging. Reconcile orders and payments before reopening affected writes.

CHAPTER 2

ARCHITECTURE DESIGN AND DEVELOPMENT PROCESS

2.1 Architecture

Overall Architecture

WashWise uses a client and cloud-service architecture. Customer screens handle registration, branches, ordering, tracking, alerts, and history. Staff screens handle the branch queue, intake verification, processing, and payment recording. Administrator screens handle staff access, branches, prices, and monitoring. These are logical role-based interfaces; the workflow does not require three separately deployed applications.

Firebase Authentication establishes identity, and Cloud Firestore stores customer and operational records. Security rules constrain permitted client data access. Trusted backend functions handle privileged actions and validate identity, role, active status, ownership, branch access, and allowed transitions. Hiding a screen is not an authorization control.

The design below specifies the target implementation of the updated workflow. Backend contracts and notification handling require implementation and validation before release.

Table 4 Technology stack

Layer	Technology and purpose
Client	React Native 0.86.3 and React 19.2.3 with TypeScript ~6.0.3.
Framework and routing	Expo ~57.0.18 and Expo Router ~57.0.21; versions reflect the supplied project configuration.
Identity and storage	Firebase JavaScript SDK ^12.19.0; Firebase Authentication for credentials and Cloud Firestore for operational records.
Branch directions	expo-location, react-native-maps, and linking support for branch location and directions.
Trusted backend	Proposed Firebase Cloud Functions and Admin SDK for staff provisioning and protected order/payment operations.
Notifications	Proposed trusted push-delivery integration with persistent in-app Firestore notifications; delivery configuration must be verified before release.

Layer	Technology and purpose
Development support	Code editor, Git repository, issue tracker, and Android test devices or emulators.

High Level Data Flow

1. Account access: customers register or log in through Authentication, and their identifiers link to users records. Staff login additionally requires an active staffAccounts record. Administrators provision staff through the protected backend.
2. Branch and order entry: customers read branch information and prices, select a branch, provide laundry details and priority, and confirm the estimate. The order stores its customer and branch identifiers and starts as pending_dropoff.
3. Physical intake: staff at the selected branch inspect the laundry and enter verified quantities and the final price. The estimate remains available for comparison. Acceptance of the laundry changes the order to received.
4. Processing and alerts: staff change the order to washing, drying, and ready as work progresses. The ready transition creates a notification event and triggers a push attempt. The customer reads only their own status and notification records.
5. Settlement and history: at collection, staff verify and record the in-person payment. A protected operation checks the ready notification, payment, and pickup confirmation before changing the order to completed. Completed orders appear in customer history.
6. Administration: the administrator maintains branch information and prices, manages staff access, and monitors operations. Staff account actions retain creator and timestamp information. Passwords never enter Firestore or audit logs.

2.2 Design

User Interface Design

The interface uses consistent navigation, readable labels, clear progress states, and visible validation. Order details identify the branch and distinguish estimated from final prices. Confirmation views explain the effect of intake verification, cancellation, payment recording, and completion. Loading, empty, and failed-network states provide a clear next action.

Customer screens comprise sign up/login, profile, branch dashboard, branch details and map, order entry, estimate confirmation, active order details, notifications, and

completed history. A progress indicator explains pending drop-off, received, washing, drying, ready, and completed using ordinary language.

Staff screens comprise login, the working-branch queue, laundry verification, processing details, ready orders, payment entry, and completion confirmation. The branch identifier is visible. Payment is recorded at collection after readiness notification and shows the final amount and responsible staff member.

Administrator screens comprise login, dashboard, staff creation, staff access control, branch/pricing management, and monitoring. The staff creation form contains only email and temporary password. Role, creator, creation time, and active status are assigned automatically. Password input is masked and never displayed again after creation.

Database Design

Firestore collections and documents provide the logical equivalent of tables and records. Document identifiers are primary keys, while identifier fields connect related records. The proposed schema follows the updated workflow; rules and trusted operations enforce relationships and permissions.

Table 5 Firestore collection schema

Collection and key	Fields and data types
users/{uid}	uid, displayName, email: strings; phone/photoURL: optional strings; createdAt, updatedAt: timestamps. Credentials remain in Authentication.
staffAccounts/{uid}	uid, email, role, createdBy: strings; role enum admin/staff; isActive: boolean; createdAt, updatedAt: timestamps; branchIds: array of strings. No password field.
branches/{branchId}	name, description, address: strings; photoURL: optional string; location: GeoPoint; isActive: boolean; pricing: array of maps containing serviceId, unit, rateMinor, and rush adjustment; updatedAt: timestamp.
orders/{orderId}	customerId, branchId, serviceId, priority: strings; laundryDetails: map; estimatedWeightKg/actualWeightKg: optional numbers; itemCount: optional integer; estimatedPriceMinor/finalPriceMinor: integers; currency: string; status: enum string; createdAt/updatedAt: timestamps.

Collection and key	Fields and data types
orders continued	pricingSnapshot: map; estimatedCompletionAt: optional timestamp; verifiedBy/paymentId: optional strings; readyAt, pickedUpAt, completedAt: optional timestamps; cancellationReason: optional string; statusHistory: array of maps with status, changedBy, and changedAt.
payments/{paymentId}	orderId, customerId, branchId, verifiedBy, currency, method, idempotencyKey: strings; amountMinor: integer; status: verified; paidAt: timestamp.
notifications/{notificationId}	customerId, orderId, type, title, message, eventKey: strings; createdAt: timestamp; readAt/lastAttemptAt: optional timestamps; deliveryStatus: pending/sent/failed; retryCount: integer.

One customer has many orders, and one branch has many orders; each order belongs to one customer and one branch. Staff identifiers connect verification, status changes, and payments to accountable users. An order has zero or one verified settlement payment in this scope and may have multiple notifications. Each notification belongs to a customer and an order. Authentication identifiers match the corresponding profile document identifiers.

Money is stored in integer minor units, such as centavos, to prevent rounding drift. Server timestamps establish the audit sequence. A pricing snapshot preserves the basis of an existing charge when branch prices change. Trusted logic checks customer-supplied quantities and recalculates estimates and final totals.

Branch assignment is a separate administrator-controlled action after staff account creation. It does not add inputs to the required email-and-temporary-password form. New staff receive no branch write access until assigned. The initial administrator is provisioned through trusted administrative setup.

API Design

The application uses Firebase SDK authentication and permitted Firestore reads under security rules. The following proposed callable functions define backend contracts; they are specifications, not a claim of deployed services. The logical endpoint is POST /{functionName} at the configured regional function host. Callable requests use a data object and successful responses use a result object; the SDK handles this envelope and attaches the Firebase identity token.

Table 6 Proposed API operations

Operation and caller	Request data	Result and validation
createStaffAccount; admin	email; temporaryPassword	uid, email, role, isActive. Verify active admin; create Authentication account and staffAccounts with automatic fields. Never return/log the password; compensate if either step fails.
setStaffAccess; admin	staffUid; isActive	Updated access state. Retain history; enforce active status on every protected request and apply Authentication disable/session controls.
assignStaffBranches; admin	staffUid; branchIds[]	Updated authorized branches. Validate branch identifiers and audit the action.
createOrder; customer	branchId; serviceId; priority; laundryDetails; requestId	orderId, pending_dropoff, estimated price, optional completion estimate. Derive customerId from identity and validate/recalculate values.
verifyLaundry; branch staff	orderId; actual weight or items	orderId, final price, received. Validate branch access, pending_dropoff, quantities, and pricing.
updateOrderStatus; branch staff	orderId; nextStatus; requestId	Updated status. Permit received to washing to drying to ready. Persist the ready notification before collection can complete.
recordPaymentAndC omplete; branch staff	orderId; amountMinor; method; pickupConfirmed; requestId	paymentId, completed. Require ready, a saved ready notification, correct final payment, and pickup confirmation. Commit settlement atomically.
cancelOrder; authorized user	orderId; optional reason	cancelled. Apply ownership/branch access and the approved cancellation policy; retain audit history.

Example request data for createOrder: {"branchId":"branch_01","serviceId":"wash_fold","priority":"regular","laundryDetails":{"estimatedWeightKg":5},"requestId":"unique-request"}. The response contains an order identifier, pending_dropoff status, and server-calculated estimate. These values illustrate the contract only.

Structured error codes include unauthenticated, permission-denied, invalid-argument, not-found, failed-precondition, and already-exists. The app shows useful messages without internal details. Request identifiers make retries safe; transactions prevent

duplicate settlement. A push failure does not undo ready status: the notification remains available in the app and its delivery can be retried.

Read operations include own customer profile, branches, own orders/notifications, the authorized staff branch queue, and administrator monitoring. Queries must match security rules and use indexes and pagination. Public access must not expose passwords, unrestricted customer records, or role administration.

2.3 Development Process

Development Methodology

The project will use iterative development. Each iteration selects approved requirements, designs the interface and data changes, implements the feature, tests it, and presents a demonstration for review. Feedback updates the backlog. Requirements and the workflow serve as the acceptance basis.

The first iteration covers authentication, roles, and branches; the second covers estimates, ordering, and intake; the third covers status updates, ready alerts, and settlement. The final iteration integrates the system, resolves defects, evaluates security and usability, and prepares release and support materials.

Development Timeline

Figure 1 Proposed full development Gantt chart

Activity	W1	W2	W3	W4	W5	W6	W7	W8
Requirements and workflow	X	X						
Architecture UI and schema		X	X					
Accounts roles and branches			X	X				
Orders estimates and intake				X	X			
Status alerts and settlement					X	X		
Integration and security tests						X	X	
Client UAT and defect fixes							X	X
Release training and handover								X
Documentation and reviews	X	X	X	X	X	X	X	X

X indicates planned activity. Week 1 begins on the approved start date. This eight-week baseline covers the full cycle and does not represent completed work. At each

weekly review, the team will update actual dates, progress, dependencies, and revised release expectations.

Version Control and Branching Strategy

The team will use a shared Git repository. The main branch contains reviewed, releasable work; a development branch integrates accepted features. Short-lived feature and fix branches address individual requirements or defects. Descriptive commits and pull requests reference issue identifiers, and release tags identify the exact application, rules, and backend versions deployed.

Credentials, service account keys, customer exports, and generated builds must not be committed. Configuration examples document variable names without secret values. The lockfile is retained for repeatable dependencies, and schema/rule changes are reviewed alongside affected code.

Code Review Procedures

Each pull request states the change, linked requirement, validation, and relevant screenshots. Another team member reviews correctness, role and branch restrictions, error handling, maintainability, and user behavior. Checks and review comments must be resolved before merging. The author updates affected documentation and test cases.

CHAPTER 3

TESTING DEPLOYMENT AND MAINTENANCE

3.1 Testing

Testing Strategy

Testing will combine unit tests for calculations and validation; integration tests for Authentication, Firestore, and backend operations; security-rule tests; and complete workflow tests. Manual device checks cover readability, permissions, network interruption, and notifications. Regression tests verify affected core scenarios after changes.

A dedicated test environment will contain separate customers, an administrator, active/inactive staff, at least two branches, and every order state. Each execution records version, device, network conditions, inputs, actual result, evidence, and pass/fail status. The following cases define expected results; execution and client approval remain pending.

Table 7 Key test cases and expected outcomes

ID and scenario	Expected outcome
TC01 Customer account	Valid signup creates an owned profile; duplicate/invalid data is rejected. A customer cannot assign privileged roles.
TC02 Staff provisioning	An active admin submits only email and temporary password. Authentication and staffAccounts receive correct automatic fields; Firestore/logs contain no password. Non-admin calls are denied.
TC03 Staff deactivation	Inactive staff cannot perform protected operations, including from existing sessions. Only an admin can reactivate access.
TC04 Branch and estimate	Selected branch, service, priority, and predefined quantities produce the correct estimate. Invalid quantities are rejected.
TC05 Order ownership	Submission starts pending_dropoff and uses the authenticated customer. Other customers cannot read/change the order.
TC06 Intake verification	Assigned branch staff verify quantities/final price and set received. Another branch's staff are denied.
TC07 Status sequence	received, washing, drying, ready proceed in order. Unauthorized, skipped, or reversed transitions are rejected.
TC08 Ready alert	Ready creates one notification event and attempts push delivery before pickup. Failed/denied push retains the in-app alert and retry status.

ID and scenario	Expected outcome
TC09 Settlement	Correct final payment plus pickup confirmation completes a ready order. Wrong amount, missing pickup, or an earlier status cannot complete it.
TC10 Retry/concurrency	Duplicate or concurrent settlement requests create one verified payment and one consistent completed order.
TC11 History	Customers see only their completed history; staff see authorized branch records; administrators monitor permitted operational records.
TC12 Cancellation	Only authorized cancellation of an eligible order succeeds. The actor/time and optional reason are retained; history/payments are not erased.
TC13 Map/network	Directions use the correct branch location. Denied location permission retains usable address information. Network errors never report false success.
TC14 Quality targets	Recorded performance/usability checks meet the proposed targets, and assistance or defects are documented.
TC15 Recovery/regression	Staging rollback preserves consistent order/payment links, and critical workflows still pass after the change.

Bug Tracking and Issue Management

A shared issue tracker records identifier, summary, reporter, affected version/environment, reproduction steps, expected/actual behavior, evidence, severity, owner, and status. Issues move through Open, Assigned, In Progress, Ready for Retest, and Closed; failed retests reopen the issue.

Unauthorized access, lost orders, and incorrect or duplicate payment records are critical and block release. Other issues are prioritized by workflow impact and client agreement. Closure requires retest evidence and a regression check where relevant. Screenshots and logs exclude unnecessary personal data; known limitations appear in release notes.

3.2 Deployment

Deployment Steps

1. Prepare the release candidate: freeze approved scope, resolve blocking defects, complete required tests, and record the release tag, dependency versions, rules, indexes, functions, and configuration.

2. Configure the target environment: confirm the Firebase project, Authentication settings, Firestore rules/indexes, trusted backend, notification credentials, and map configuration. Provision the administrator securely, validate branch data, and authorize staff assignments.
3. Back up and deploy to staging: retain the previous release and configuration and back up data before risky changes. Deploy compatible rules, indexes, and backend functions, then produce the Expo mobile test build.
4. Run staging verification and user acceptance: exercise all roles and the complete workflow, security boundaries, notifications, retries, and payment reconciliation. Fix blocking issues, train client representatives, and record their actual acceptance decision.
5. Release through the agreed distribution channel: publish the approved build or authorized installation package with its compatible backend configuration. Retain the previous working release and the recovery instructions.
6. Verify after release: check login, branches, order creation, processing updates, ready alerts, settlement, and history. Monitor errors, record deployment results, and provide release notes and support guidance.

Environment and Configuration Management

Table 8 Deployment environments

Environment	Configuration and control
Development	Developer builds with synthetic accounts and an isolated Firebase project or emulators. No real client transactions.
Staging	A separate project/configuration matching production rules, indexes, functions, and notification behavior. Approved test data supports integration, security, recovery, and acceptance testing.
Production	Client-approved configuration and real operations. Restrict administration, retain audit records/backups, monitor failures, and deploy only approved versioned releases.

Each environment uses distinct project identifiers, endpoint host/region, application configuration, and access controls. Secrets are kept in protected configuration services and never embedded as privileged credentials in the mobile application. Public client configuration does not replace authentication and authorization.

Rollback Procedures

The release owner initiates rollback for security failures, inconsistent orders/payments, widespread crashes, or broken critical workflows. Restrict affected writes when necessary, preserve logs and a current snapshot, and inform the client representative.

Restore the last verified compatible application and backend configuration. Prefer backward-compatible schema changes. Restore data only after evaluating transactions created since the backup; reconcile recent orders/payments rather than overwriting valid activity blindly. Retest critical scenarios before resuming full access and document the incident and corrective action.

User Acceptance Letter and Agreement

This form is completed after client testing. An unsigned form does not establish acceptance.

Date: _____

To: The WashWise Project Development Team

Subject: User Acceptance of the WashWise Application

I, _____, authorized representative of _____, acknowledge that WashWise version _____ was presented for user acceptance testing on _____. The reviewed scope covers customer accounts, branches, ordering and estimates, staff verification, processing updates, ready notifications, in-person payment, collection, and history.

The attached test record identifies scenarios, results, unresolved issues, and agreed conditions. The application excludes online payment, delivery, customer-controlled pickup scheduling, and automated machine control.

Acceptance decision: Accepted / Accepted subject to conditions / Not accepted.

Conditions or unresolved issues:

Corrective actions and target dates:

Client representative name and position:

Client signature: _____ Date: _____

Development team representative:

Team signature: _____ Date: _____

3.3 Maintenance

Maintenance and Support Plan

During the agreed support period, a designated maintenance owner will monitor authentication/application errors, failed alerts, and order/payment consistency. Critical security and payment incidents receive immediate triage; routine defects and improvements enter the backlog. The client and team will agree on support coverage and response commitments at handover.

Weekly reviews examine open issues, failed notifications, staff/branch access, and backup results. Monthly reviews assess dependencies, device compatibility, recurring errors, and documentation. Changes affecting roles, rules, calculations, data structures, or state transitions require staging validation and a recovery plan.

The administrator deactivates departing staff promptly while preserving history. Backups follow the agreed schedule, and restoration exercises verify usability. Diagnostics collect only information needed to resolve an issue.

Documentation Updates

Accepted changes update the requirements, screen descriptions, schema, API contracts, tests, deployment procedures, and user guide. Release notes identify version, date, changes, known issues, and operator actions. Approved documentation is stored with the corresponding release, while prior versions remain traceable.

User Support Channels and Resources

Customers use an in-app help/FAQ resource and the branch's published contact details. Staff report operational problems to the administrator, who escalates application defects to the designated development support contact. Reports include order identifier, time, device, and description, but never a password.

Handover resources include customer/staff guides, administrator access instructions, troubleshooting and recovery guides, release notes, and agreed support contacts. Training demonstrates the complete workflow, estimates versus final prices, failed notifications, and disputed records.

CONCLUSION

Summary

WashWise is proposed to coordinate branch selection, laundry orders, staff verification, processing visibility, ready notifications, and recorded in-person settlement. Customers physically drop off and collect laundry. Administrators control staff access, branches, and pricing; authorized staff handle branch operations.

The requirements and plans provide an implementation and evaluation basis. Benefits must be established through actual testing and client use. Release readiness requires verified workflow behavior, access controls, payment consistency, and client acceptance.

Future Work

Future improvements may include operational reporting, completion estimates informed by recorded workload, accessibility refinements, and broader device support. Additional services require separate scope approval, revised design, and acceptance tests.

REFERENCES

- Leong, Y. M., Ku Azir, K. N. F., Ibrahim, S. Z., & Azemi, S. N. (2020). LaundryMama: Humanising laundry tasks using laundry management system and laundry-on-demand mobile applications. *IOP Conference Series: Materials Science and Engineering*, 767(1), 012061. <https://doi.org/10.1088/1757-899X/767/1/012061>
- Nurmayangsari, K. D., & Widiono, S. (2025). Transformasi digital layanan laundry melalui mobile tracking untuk meningkatkan transparansi dan efisiensi operasional UMKM. *Edumatic: Jurnal Pendidikan Informatika*, 9(3), 885-894. <https://doi.org/10.29408/edumatic.v9i3.32601>
- Rosli, M. F., Ibrahim, S. Z., Karim, M. N. A., Mat, M. H., Noh, F. H. M., & Inthasuth, T. (2026). An intelligent on-demand laundry logistics platform with real-time location tracking and machine learning-based customer support. *International Journal of Advanced Computer Science and Applications*, 17(8). <https://doi.org/10.14569/IJACSA.2026.0170830>
- Expo. (2026). Expo SDK reference (SDK 57). <https://docs.expo.dev/versions/v57.0.0/>
- Firebase. (n.d.). Cloud Firestore documentation. <https://firebase.google.com/docs/firestore>
- Firebase. (n.d.). Call functions from your app. <https://firebase.google.com/docs/functions/callable>